

# SDE Intern Assignment



## SDE Intern Assignment

### Instructions:

- Please **choose any 1 of the following 4** problem statements for your submission
- Read the **submission guidelines** and **evaluation criteria** carefully for the chosen problem statement as mentioned below.
- **Submission Timeline:** You have a total of **3 days** to complete the assignment. The solution needs to be submitted by the **4th day** after receiving the problem statement.

### For Problem Statements 1-4:

#### **Submission Instructions:**

- Submit a working backend demo (local with ngrok or simple cloud instance).
- Provide a GitHub repository containing your code.
- Include a README.md with setup instructions, architecture, and API usage examples.
- Provide sample curl/Postman requests to test your endpoints.
- Submit a short screen recording showing your endpoints working with sample inputs.

#### **Evaluation Criteria:**

- Correctness of API responses and adherence to JSON schemas.
- Handling of both text and image inputs with OCR.
- Implementation of guardrails and error handling.
- Code organization, clarity, and reusability.
- Effective use of AI for chaining and validation.

# Problem Statement 1: AI-Powered Appointment

# **Problem Statement 1: AI-Powered Appointment Scheduler Assistant**

**Focus Area:** OCR -> Entity Extraction -> Normalization

## **Problem Statement:**

Build a backend service that parses natural language or document-based appointment requests and converts them into structured scheduling data. The system should handle both typed text and noisy image inputs (e.g., scanned notes, emails). Interns must design a pipeline with OCR, entity extraction, normalization, and final structured JSON output with guardrails for ambiguity.

## **Step 1 - OCR/Text Extraction**

Handle typed requests or photos of notes/emails.

Input (text):

Book dentist next Friday at 3pm

Input (image -> OCR sample):

book dentist nxt Friday @ 3 pm

Expected Output (JSON):

```
{
  "raw_text": "Book dentist next Friday at 3pm",
  "confidence": 0.90
}
```

## **Step 2 - Entity Extraction**

Extract date/time phrase and department.

Expected Output (JSON):

```
{
  "entities": {
    "date_phrase": "next Friday",
    "time_phrase": "3pm",
    "department": "dentist"
  },
  "entities_confidence": 0.85
}
```

### Step 3 - Normalization (Asia/Kolkata)

Map phrases to ISO date/time in local timezone.

Expected Output (JSON):

```
{
  "normalized": {
    "date": "2025-09-26",
    "time": "15:00",
    "tz": "Asia/Kolkata"
  },
  "normalization_confidence": 0.90
}
```

Guardrail / Exit Condition (JSON):

```
{"status": "needs_clarification", "message": "Ambiguous date/time or department"}
```

### Step 4 - Final Appointment JSON

Combine entities and normalized values.

Expected Output (JSON):

```
{
  "appointment": {
    "department": "Dentistry",
    "date": "2025-09-26",
    "time": "15:00",
    "tz": "Asia/Kolkata"
  },
  "status": "ok"
}
```

## Problem Statement 2: AI-Powered Health Risk

# **Problem Statement 2: AI-Powered Health Risk Profiler**

**Focus Area:** OCR -> Factor Extraction -> Risk and Recommendations

## **Problem Statement:**

Develop a service that analyzes lifestyle survey responses (typed or scanned forms) and generates a structured health risk profile. The intern must handle noisy inputs, missing answers, and provide guardrails for incomplete data. Output must include factors, risk level, and actionable recommendations.

## **Step 1 - OCR/Text Parsing**

Ingest survey form (text/image) and parse key fields.

Input (text):

```
{"age":42,"smoker":true,"exercise":"rarely","diet":"high sugar"}
```

Input (image -> OCR sample):

```
Age: 42
Smoker: yes
Exercise: rarely
Diet: high sugar
```

Expected Output (JSON):

```
{
  "answers":{"age":42,"smoker":true,"exercise":"rarely","diet":"high sugar"},
  "missing_fields": [],
  "confidence": 0.92
}
```

Guardrail / Exit Condition (JSON):

```
{"status":"incomplete_profile","reason": ">50% fields missing"}
```

## **Step 2 - Factor Extraction**

Convert answers into risk factors.

Expected Output (JSON):

```
{
  "factors": ["smoking", "poor diet", "low exercise"],
  "confidence": 0.88
}
```

### Step 3 - Risk Classification

Compute risk level with simple scoring logic (non-diagnostic).

Expected Output (JSON):

```
{
  "risk_level": "high",
  "score": 78,
  "rationale": ["smoking", "high sugar diet", "low activity"]
}
```

### Step 4 - Recommendations

Generate actionable, non-diagnostic guidance.

Expected Output (JSON):

```
{
  "risk_level": "high",
  "factors": ["smoking", "poor diet", "low exercise"],
  "recommendations": ["Quit smoking", "Reduce sugar", "Walk 30 mins daily"],
  "status": "ok"
}
```

## Problem Statement 3: AI-Powered Report Simplifier

# **Problem Statement 3: AI-Powered Medical Report Simplifier**

**Focus Area:** OCR -> Test Extraction -> Plain-Language Explanation

## **Problem Statement:**

Create a backend service that takes medical reports (typed or scanned) and produces patient-friendly explanations. Interns must handle OCR errors, normalize tests, and ensure no hallucinated results are added. Final output must include normalized tests and simplified explanations.

## **Step 1 - OCR/Text Extraction**

Extract test names, values, units; fix minor typos.

Input (text):

CBC: Hemoglobin 10.2 g/dL (Low), WBC 11,200 /uL (High)

Input (image -> OCR sample):

CBC: Hemoglobin 10.2 g/dL (Low)  
WBC 11200 /uL (Hgh)

Expected Output (JSON):

```
{
  "tests_raw": [
    "Hemoglobin 10.2 g/dL (Low)",
    "WBC 11200 /uL (High)"
  ],
  "confidence": 0.80
}
```

## **Step 2 - Normalized Tests JSON**

Standardize names, units, ranges, and statuses.

Expected Output (JSON):

```
{
  "tests": [

    {"name": "Hemoglobin", "value": 10.2, "unit": "g/dL", "status": "low",
    "ref_range": {"low": 12.0, "high": 15.0}},

    {"name": "WBC", "value": 11200, "unit": "/uL", "status": "high", "ref_
    range": {"low": 4000, "high": 11000}}

  ],
  "normalization_confidence": 0.84
}
```

### Step 3 - Patient-Friendly Summary

Explain findings simply without diagnosing.

Expected Output (JSON):

```
{
  "summary": "Low hemoglobin and high white blood cell count.",
  "explanations": [
    "Low hemoglobin may relate to anemia.",
    "High WBC can occur with infections."
  ]
}
```

Guardrail / Exit Condition (JSON):

```
{"status": "unprocessed", "reason": "hallucinated tests not present
in input"}
```

### Step 4 - Final Output

Return combined normalized tests and summary.

Expected Output (JSON):

```
{
  "tests": [

    {"name": "Hemoglobin", "value": 10.2, "unit": "g/dL", "status": "low",
    "ref_range": {"low": 12.0, "high": 15.0}},

    {"name": "WBC", "value": 11200, "unit": "/uL", "status": "high", "ref_
    range": {"low": 4000, "high": 11000}}

  ],
  "summary": "Low hemoglobin and high white blood cell count.",
  "explanations": [
    "Low hemoglobin may relate to anemia.",
    "High WBC can occur with infections."
  ]
}
```

```
],  
  "summary": "Low hemoglobin and high white blood cell count.",  
  "status": "ok"  
}
```

## Problem Statement 4: AI-Powered Amount Detection

# **Problem Statement 4: AI-Powered Amount Detection in Medical Documents**

**Focus Area:** OCR -> Numeric Normalization -> Context Classification

## **Problem Statement:**

Design a service that extracts financial amounts from medical bills or receipts (typed or scanned, possibly crumpled or partially visible). Interns must handle OCR errors, digit corrections, classification by context, and produce final structured JSON with provenance.

## **Step 1 - OCR/Text Extraction**

Extract raw numeric tokens from bills/receipts.

Input (text):

Total: INR 1200 | Paid: 1000 | Due: 200 | Discount: 10%

Input (image -> OCR sample):

T0tal: Rs 1200 | Pald: 1000 | Due: 200

Expected Output (JSON):

```
{
  "raw_tokens": ["1200", "1000", "200", "10%"],
  "currency_hint": "INR",
  "confidence": 0.74
}
```

Guardrail / Exit Condition (JSON):

```
{"status": "no_amounts_found", "reason": "document too noisy"}
```

## **Step 2 - Normalization**

Fix OCR digit errors and map to numbers.

Expected Output (JSON):

```
{
  "normalized_amounts": [1200, 1000, 200],
  "normalization_confidence": 0.82
}
```

### Step 3 - Classification by Context

Use surrounding text to label amounts.

Expected Output (JSON):

```
{
  "amounts": [
    {"type": "total_bill", "value": 1200},
    {"type": "paid", "value": 1000},
    {"type": "due", "value": 200}
  ],
  "confidence": 0.80
}
```

### Step 4 - Final Output

Return labeled amounts with currency and provenance.

Expected Output (JSON):

```
{
  "currency": "INR",
  "amounts": [
    {"type": "total_bill", "value": 1200, "source": "text: 'Total: INR 1200'"},
    {"type": "paid", "value": 1000, "source": "text: 'Paid: 1000'"},
    {"type": "due", "value": 200, "source": "text: 'Due: 200'"}
  ],
  "status": "ok"
}
```